A New Vertical Edge Detection Algorithm and its Application

Abbas M. Al-Ghaili, Syamsiah Mashohor, Alyani Ismail, and Abdul Rahman Ramli
Department of Computer and Communication Systems, Faculty of Engineering, Universiti Putra Malaysia
abbasghaili@yahoo.com.my. syamsiah,alyani,arr@eng.upm.edu.my

***ABSTRACT*-Edge detection is a very important process for many image processing applications, especially in Car License Plate Detection and Recognition Systems (CLPDRS). The need to distinguish the desired details is a very important pre-process in order to give good results in short time processing. We proposed a new and fast Vertical Edge Detection Algorithm (VEDA) which is based on the contrast between the gray scale values. Once, input gray image was binarized by using adaptive threshold, Unwanted Lines Elimination Algorithm (ULEA) was proposed and applied. After that, a VEDA was applied for experimental images. Then, implementation on the application is performed and discussed in order to confirm that VEDA is robust for highlighting license plate details easily. The results revealed accurate edge detection performance and demonstrated the great efficiency of using VEDA in order to highlight license plate details. Finally, VEDA showed that it is faster than Sobel operator by about 7-9 times.**

I. INTRODUCTION

Edge detection is an important issue in image processing. Edges constitute a significant portion of the information contained in images, and thus, it is useful to extract these features from a complete image. An edge map has vastly reduced complexity and retains important structure present in the original image [1].

Historically, the most common and earliest edge detection algorithms are those based on the gradient, such as the Sobel operator [2] and the Roberts operator [3].

Vertical Edge Extraction (VEE) is a basic process in CLPDRS where VEE can display car plate characters and details; therefore, it is useful to be used in order to distinguish these details.

Literature showed that VEE had been used for license plate detection such as [4] [5] [6]. VEE is one of the most crucial steps in Car License Plate Detection (CLPD), and influences the whole system in locating the plate efficiently [7].

Car license plate details always have a lot of vertical edges that can be used to generate the candidate regions for classification [4].

Most edge detectors do not perform well on images that have been blurred and do not contain “sharp” edges. For this reason, it is often useful to pre-filter an image prior to performing edge detection [1].

To detect car license plate automatically, many difficulties have to be overcame, such as the background details, the poor image quality, the processing time, the plate sizes and designs.

Nowadays, Automatic Car License Plate Recognition (ACLPR) becomes a key technique to many automated transport systems. CLPD is a basic and important stage in this technique, and it received considerable attention. Vertical edge-based CLPD is the most suitable technique to overcome some of the above mentioned difficulties, because of the abundance availability of vertical edges. Therefore, vertical edge-based CLPD systems need more improvements in order to solve those difficulties.

Fast detection of car plate vertical edges did not receive enough attention. This paper aims to propose a new and fast algorithm for detecting vertical edges.

Sobel operator is one of the most important algorithms that has been widely used to detect the vertical edges in many CLPD methods [8] [9] [10]. Therefore, this paper proposes a Vertical Edge Detection Algorithm (VEDA) to be faster than Sobel operator with an accurate performance of showing the vertical edges. Furthermore, the use of VEDA in CLPD would enhance the system performance.

It is known that a license plate consists of some characters and numbers, and it can be considered that it is a text-region [11] [12]. Therefore, localizing and detecting text-regions means the localizing of the license plates in such CLPD systems. Thus, if an algorithm efficiently localizes license plate regions would be a key algorithm to be used in CLPD approaches as a successful algorithm for detecting the vertical edges. So, that would help to detect the plate details in the scene image. This paper confirms that VEDA can detect the vertical edges in the license plate region. Moreover, the application discussed in this paper (section 3), highlights the vertical edges (VEDA output). This proves that VEDA is able to work with too blurry images caused by motion and it also can detect the vertical edges for the blurry input images.

This paper is organized as follows. Section II describes our algorithm of vertical edge detection, and it contains two parts: Unwanted Lines Elimination Algorithm (ULEA) process and VEDA process. Section III gives an application for using VEDA. Experimental results and discussion are given in section IV. Section V draws the conclusion.

II. THE PROPOSED VERTICAL EDGE DETECTION ALGORITHM

A. Overview

We first apply adaptive thresholding [13] on image, then, apply ULEA in order to remove noise and enhance the binarized image, and finally, extract the vertical edges by

using VEDA, then, when comparing the VEDA with Sobel operator; we found that it was faster.

The following diagram shows the flowchart for our proposed approach as illustrated in Fig. 1.

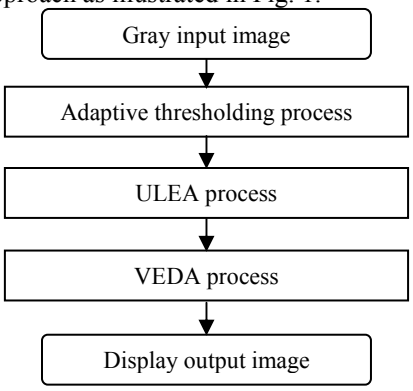


Fig. 1: The flowchart of proposed approach

B. Adaptive Thresholding Process

The input image is gray scale and has the values between [0-255]. Bradley's method in [13] is applied on the input image in order to form the binarized image. The researchers in [13] have recently proposed real-time adaptive thresholding using mean of a local window [14], where local mean is computed using integral image. Fig. 2 shows the input image and the result of applying adaptive thresholding.

Fig. 2 (a): Input image

Fig. 2 (b): Thresholded image

C. ULEA Process

Thresholding process in general will produce many thin lines which do not belong to car plate region. So elimination of these lines will contribute in CLPDRS accuracy and reduce the processing speed. Therefore, an algorithm is proposed in order to eliminate these unwanted lines. It is clear from Fig. 2 (b) that the image has some unwanted lines in angles 0^0, 90^0, 45^0, and 135^0 with width of one pixel. Therefore, ULEA has been proposed in order to eliminate these lines. In order to keep small plate details, one pixel width is selected to be used for this process. A 3x3 mask is used throughout all image pixels. Only black pixel values in the thresholded image are tested. Supposed that g(x,y) are the values for thresholded image. Once, the current pixel value located at center of the mask (x,y) is black, the 8-neighbor pixel values are tested. If two corresponding values are white in the same time, then the current pixel is converted to white value as background pixel.

Fig. 3 shows the possible cases in which the current pixel is converted to background pixel. Each case of a,b,c and d represents two corresponding values in each time the mask moves through the g(x,y) values.

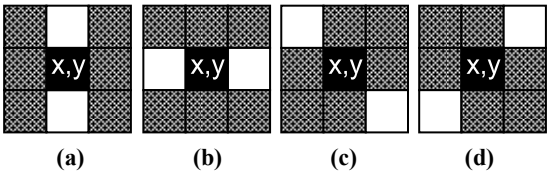


Fig. 3: The 4-cases for converting center pixel to background value, **(a)** for horizontal lines (0°), **(b)** for vertical lines (90°), **(c)** for inclined lines with (45°), **(d)** for inclined lines with (135°)

The pseudocode for this process can be summarized as illustrated in Algorithm 1, while Fig. 4 shows the output after applying ULEA process on Fig. 2 (b).

```
For every pixel in thresholded image
  If(g(x,y)=0)
        If[ (g(x-1,y)=255 AND g(x+1,y)=255)
          OR
            (g(x,y-1)=255 AND g(x,y+1)=255)
          OR
            (g(x-1,y+1)=255 AND g(x+1,y-1)=255)
          OR
            (g(x-1,y-1)=255 AND g(x+1,y+1)=255) ]
        Then
        O(x,y)=255;
  End if
End for
```

Algorithm1: ULEA pseudocode

Fig. 4: ULEA output

D. VEDA Process

The advantage of VEDA is to distinguish the plate details region, especially, the beginning and the end of each character therefore, the plate details will be easily detected and character recognition process will be done faster. VEDA will make the black-white and the white-black regions edges

have two and one black pixels, respectively. A mask 2X4 is proposed for this process, where (x,y) represents the current processed pixel location at point (0,1) as center of our mask as shown in Fig. 5 (a). Basically, the proposed mask consists of 3 smaller masks, which are left mask 2X1, center mask 2X2, and right mask 2X1. Fig. 5 shows the design of the proposed mask, where x represents the rows or the height of the image and y represents the columns or the width of the image.

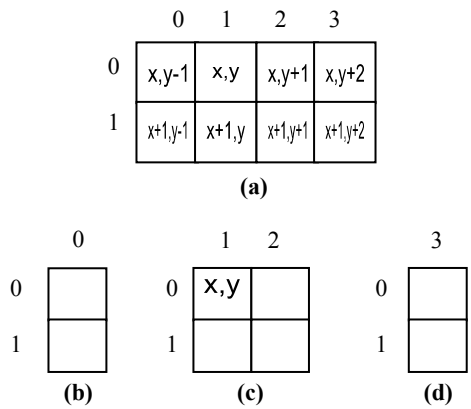


Fig. 5: The design of the proposed mask:
(a) moving mask, **(b)** left mask (0,0), (1,0), **(c)** center mask (0,1), (0,2), (1,1), (1,2), **(d)** right mask (0,3), (1,3)

The selection of the column at the locations (0,1) and (1,1) to be a center of our proposed mask, is to keep the last two black pixel values located at the intersection of black-white regions, and to keep the first black pixel value located at the intersection of white-black regions as well.

The 2X4 mask moves from top to bottom and from left to right. The abundance availability of black pixel values inside the plate itself and because plate characters are white color which is considered as background in our work, therefore, our proposed mask can keep the last two black pixel values, which are located at the end of intersection of plate region with the beginning of the character (black-white regions) as shown in Fig. 6 (a) [15] [16]. Similarly, the mask can keep the first black pixel value that is located at the end of the character in the plate itself (white-black regions) as shown in Fig. 6 (b).

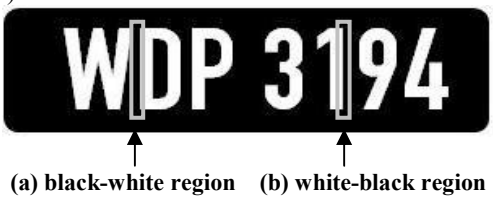


Fig. 6: The intersection of black-white and white-black areas

Actually, this process is done for both of the edges left and right. The first edge and/or (odd edges) can be two black pixels width and the second edge and/or (even edges) can be one black pixel width. The mask starts moving from top to bottom and from left to right. If the four pixels at center mask in locations (0,1), (0,2), (1,1), and (1,2) are black, then the two right mask values are tested whether they are black or not, and the two left mask values are tested whether they are black or not. If the three masks are black values, then the mask center column at (0,1) and (1,1) is converted to white and so on.

Algorithm 2 shows the pseudocode for VEDA process, while the result of the proposed algorithm on the image in Fig. 4 is shown in Fig. 7.

```
Create a white blank image as Img(x,y);
// Σ_{x=0}^{hieght} Σ_{y=0}^{width} Img(x,y) = 255 ;
For every pixel in ULEA image output
  center=1; left=1; right=1;

  If(four center mask values = black)
          center=0;
  End if
  If(two right mask values = black)
          right=0;
  End if
  If(two left mask values = black)
          left=0;
  End if

  If(!center AND !right AND !left)
    two center column values of Img =white;
    /*
      Img(x,y)=255;
      Img(x+1,y)=255;
    */
  End if
End for
```

Algorithm2: VEDA pseudocode

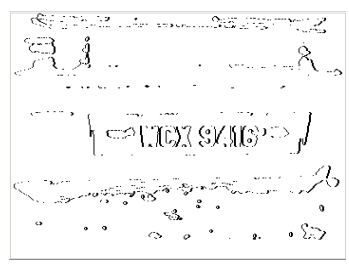


Fig. 7: VEDA output

III. Application for using VEDA

A. Highlight desired details based on vertical edge detection

After Vertical Edge Detection Stage is done, filling white-regions which are located between each two vertical edges. Because of the vertical edge output design, one edge has two-pixel width and the other has one pixel width, it is easy to track the wanted edges and make connection between each two vertical edges which probably located in plate region.

All the pixels in vertical edge step image will be scanned, when two neighbor black pixels and followed by one black pixel, the horizontal distance (hd) is calculated and must satisfy: $1<hd<33$, where *hd* represents the length between the two vertical edges. Actually, this criterion can remove very

short lines and long lines that do not belong to plate details regions. This scanning process will start moving from right to left and from top to bottom. After all pixels are scanned, the output will result white image except the regions which probably contain plate details.

The pseudocode for this process can be summarized as illustrated in Algorithm 3, while Fig. 8 shows the output after applying this algorithm on Fig. 7.

```
For all pixels in Fig. 7, read each two neighbor pixels (p1(r,c), p2(r,c+1));
 If they are black (p1(r,c), p2(r,c+1))
  If {(the next single pixel is black (pn(r,c+n)) AND (1<hd<33)
        AND (all pixel values between pn, p2 are white)}
   For the rows (in thresholded output image) (r-1, r-2, r-3)
    For the cols (in thresholded output image) (c, c-1, c-2, c-3, c-4)
     calculate how many black pixels are, and store it in bp;
        If(bp ≤ 2)
           it is the left-upper corner of plate
           draw black line between (p2, pn);  //in Img4
        Else if(p1(r,c), p2(r,c+1),p3(r,c+3),p4(r,c+4) are black)
           draw black line between (p2, pn);  //in Img4
        Else if(p3(r,c+3), p4(r,c+4) are black)
           draw black line between (p2, pn);  //in Img4
        End If
    End For
   End For
  End If
 End If
End For
```

Algorithm3: Highlight desired details pseudocode

Fig. 8: Highlight desired details output

IV. Experimental results & Discussions

In this section we compare VEDA in terms of the computational time and visual result for vertical edge detection stage, with Sobel detection method. The tested images contain vertically and horizontally blurred images and diagonal images as well. Two datasets are tested. The dataset1 is a 352X288 as shown in Figs. 10-12. The dataset2 is a 640X480 as shown in Fig. 9. Dataset1 consists of 30 images while dataset2 consists of 20 images. Both datasets are tested on laptop Core 2 CPU 1.83GHz, 1 GB of RAM. Dataset1 images were captured by using a web-camera with 30 fps. The program is running under Windows XP SP2, written in C++ language using OpenCV library [17] [18] [19].

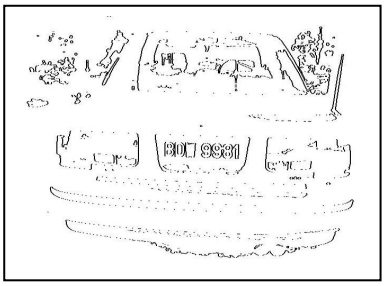
Results of running Sobel operator for vertical edge detection and VEDA respectively on a sample image are shown in Fig. 9 (b) and 9 (c).

Table I shows the experimental results in terms of processing time for Sobel operator and VEDA for two datasets.

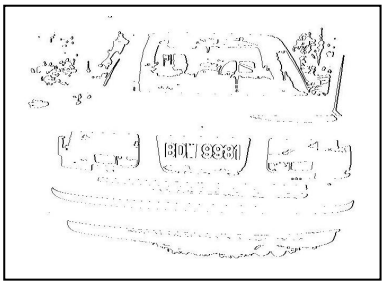
Two examples of dataset1 are shown in Figs. 10 and 11. Fig. 10 shows that VEDA can detect inclined vertical edges efficiently, while Fig. 11 gives a comparison between Sobel and VEDA outputs for a blurry image.

An example is shown in Fig. 12, in order to demonstrate that VEDA output can reserve license plate details. Therefore, those details can be extracted and highlighted easily for further use.

(a) Input image



(b) Sobel's result (time=130msec)



(c) Our algorithm's result (time=15msec)

Fig. 9: Result of applying Sobel and VEDA.
Note that VEDA ≈ 9 times faster than Sobel's

From Fig. 9, we can notice that car license plate vertical edges are distinguished and clear to be extracted easily, the two pixel width and the one pixel width are clear too. It is important to say the first edge of the plate character is darker than the second one. The computational time in VEDA is much less than Sobel which gives a strong advantage to VEDA.

Table I: The comparison between Sobel and VEDA

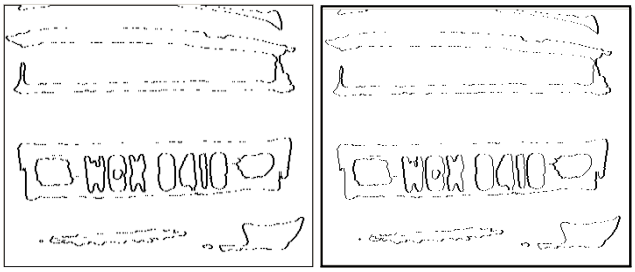
Methods	Datasets	Time (ms)
Sobel operator	Dataset 1	47
	Dataset 2	130
VEDA	Dataset 1	7
	Dataset 2	15

From Table I, we can see that VEDA outperforms over Sobel algorithm, in term of processing time. ULEA also contributes to faster VEDA performance with reduction of the total details that need to be processed.

Fig. 10: Vertical edges detection using our proposed approach

From Fig. 10, it is noticeable that VEDA is robust to detect inclined vertical edges.

(a)

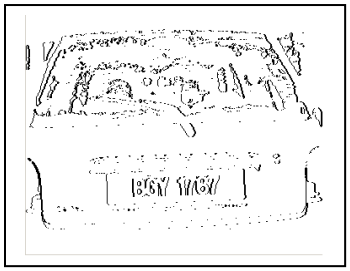


(b) **(c)**

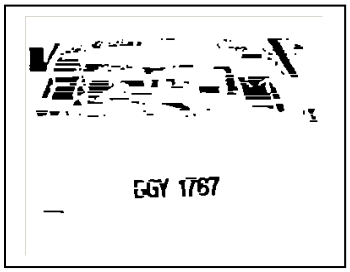
Fig. 11: Blurry input image and its Sobel and VEDA outputs: (a) Blurry input image; Sobel (b); and VEDA (c) outputs for input blurry image

From Fig. 11, we can see that our proposed approach is able to work with high blurry images caused by motion and also it can detect the vertical edges for blurry.

(a)



(b)



(c)

Fig. 12: Highlight desired details based on VEDA, **(a)**: input image, **(b):** VEDA output, **and (c):** desired details detection.

From Fig. 12, the results show that the plate details are extracted correctly with some noisy region which will be removed later to detect the plate region only.

V. CONCLUSION

In this paper, we have proposed a new algorithm in order to detect vertical edges. Before VEDA was applied, ULEA was done in order to enhance the binarized image. This enhancement process makes VEDA works faster than Sobel. The experimental results demonstrated that VEDA is faster than Sobel operator for extracting vertical edges, yet the accuracy of edge detection is nearly similar. Our proposed approach is able to work with motion blurred images for detecting the vertical edges. The advantage of VEDA is that this algorithm gives two pixel width for black-white region edges and one pixel width for the white-black region edges. This advantage helps to detect plate details easily and faster during next processing steps and suitable for real-time processing.

REFERENCES

[1] R. C. Hardie and C. G. Boncelet, "Gradient-Based Edge Detection Using Nonlinear Edge Enhancing Prefilters," *IEEE Trans. Image Process.*, vol. 4, pp. 1572-1577, 1995.

[2] L. S. Davis, "A Survey of Edge Detection Techniques," *J. Computer Graphics Image Processing*, vol. 4, pp. 248-270, 1975.

[3] J. K. Aggarwal, R. O. Duda, and Rosenfeld, *Computer Methods in Image Analysis.* New York: (IEEE Press), 1977.

[4] H. Bai, J. Zhu, and C. Liu, "A Fast License Plate Extraction Method on Complex Background," *in Proc. IEEE Int. Conf. Intelligent Transportation Systems*, China, October 2003, pp. 985-987.

[5] H. Bai and C. Liu, "A Hybrid License Plate Extraction Method based on Edge Statistics and Morphology," *in Proc. 17th Int. Conf. Pattern Recognition, IEEE Computer Society*, CA, August 2004, pp. 831-834.

[6] M. Yu and Y. D. Kim, "An Approach to Korean License Plate Recognition based on Vertical Edge Matching," *in Proc. IEEE Int. Conf. Systems, Man, and Cybernetics*, USA, October 2000, pp. 2975-2980.

[7] W. Jia, H. Zhang, and Xiangjian, "Region-based License Plate Detection," *J. Network and Computer Applications*, vol. 30, pp. 1324-1333, 2007.

[8] S. Thanongsak and C. Kosin, "The Recognition of Car License Plate for Automatic Parking System," *in Proc. 5th Int. Symp. Signal Processing and Its Applications*, Australia, August 1999, pp. 455-457.

[9] D. Zheng, Y. Zhao, and J. Wang, "An Efficient Method of License Plate Location," *Pattern Recognition Lett.*, vol. 26, pp. 2431-2438, 2005.

[10] S. Thanongsak and C. Kosin, "Extracting of Car License Plate Using Motor Vehicle Regulation and Character Pattern Recognition," *in Proc. IEEE Asia-Pacific Conf. Circuit and Systems*, November 1998, pp. 559-562.

[11] K. Jung, K. I. Kim, and A. K. Jain, "Text information extraction in images and video: a survey," *J. Pattern Recognition*, vol. 37, pp. 977 – 997, 2004.

[12] D. Chen, J.-M. Odobez, and H. Bourlard, "Text detection and recognition in images and video frames," *J. Pattern Recognition*, vol. 37, pp. 595-608, 2004.

[13] D. Bradley and G. Roth, "Adaptive thresholding using the integral image," *J. Graphics Tools*, vol. 12, pp. 13–21, 2007.

[14] F. Shafait, D. Keysers, and T. M. Breuel. (2008, February 8th). "Efficient Implementation of Local Adaptive Thresholding Techniques using Integral Images," in Document Recognition and Retrieval, USA [Online]. Available: http://pubs.iupr.org/cache/2007-IUPR-11Sep_1129.pdf

[15] Wikipedia. (2008, January 3rd). Vehicle registration plates of Malaysia [Online]. Available: http://en.wikipedia.org/wiki/Malaysian_vehicle_license_plates.

[16] N. A. Jusoh, J. M. Zain, and T. A. A. Kadir, "Enhancing Thinning Method for Malaysian Car Plates Recognition," *in Proc. 2nd IEEE Int. Conf. Innovative Computing, Information, and Control*, Japan, September 2007, pp. 378-378.

[17] Intel. (2007, July 23rd). Open Source Computer Vision Library [Online]. Available: http://www.intel.com/technology/computing/opencv/index.htm

[18] Intel and SourceForge.net. (2007, July 22nd). SourceForge_net [Online]. Available: http://sourceforge.net/projects/opencvlibrary.

[19] G. Agam. (2007, July 22nd). Introduction to programming with OpenCV, Illinois Institute of Technology [Online]. Available: http://www.cs.iit.edu/~agam/cs512/lect-notes/opencv-intro/index.html